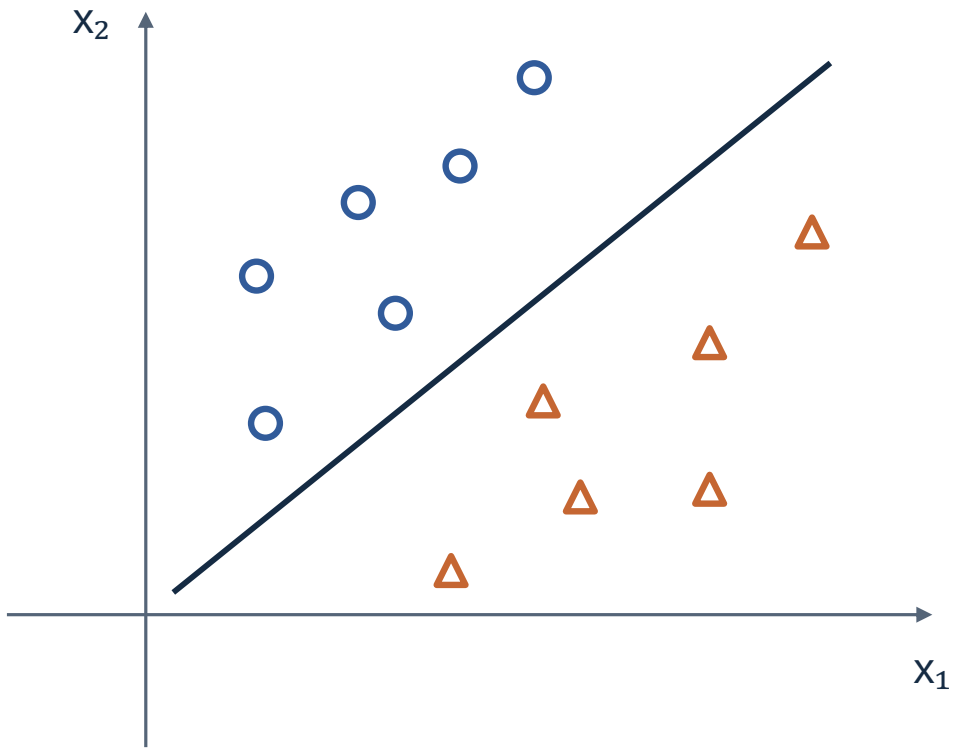
The Perceptron

Learning a linear decision boundary

One neuron.
A binary decision.
A rule that learns from mistakes.

Dr. Abdelrahman Elfikky
University of Arkansas at Little Rock



$$s = \mathbf{w}^T \mathbf{x} + b$$

Brief History of the Perceptron

1943

McCulloch & Pitts

Introduced one of the first mathematical models of an artificial neuron.

MODEL THE NEURON

1949

Donald Hebb

Proposed that neural connections become stronger when neurons are repeatedly active together.

UNDERSTAND LEARNING

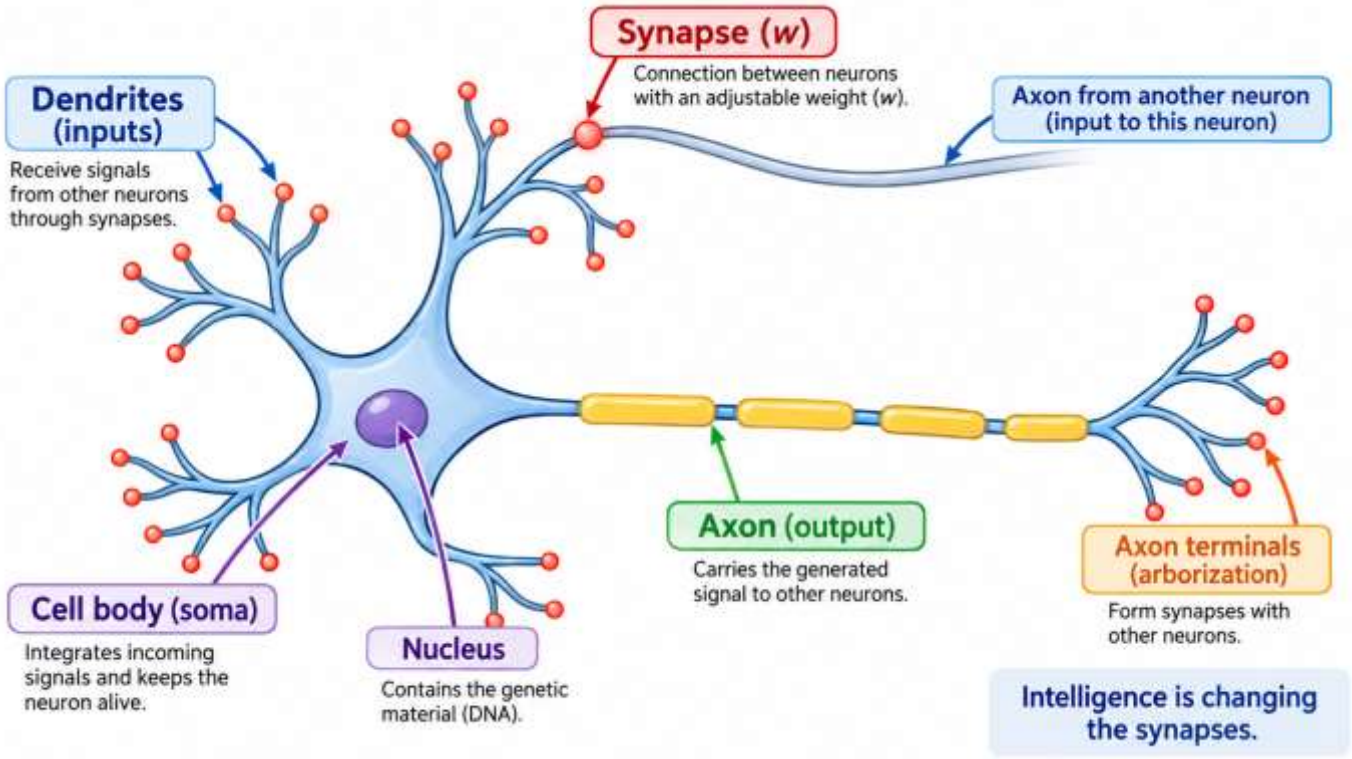
1958

Frank Rosenblatt

Introduced the Perceptron — a trainable artificial neuron that learns to classify by adjusting its weights from errors.

BUILD A NEURON THAT LEARNS

Biological inspiration not a literal brain model



Dendrites

Inputs arrive.

Synaptic connections

Connections have strengths.

Cell body

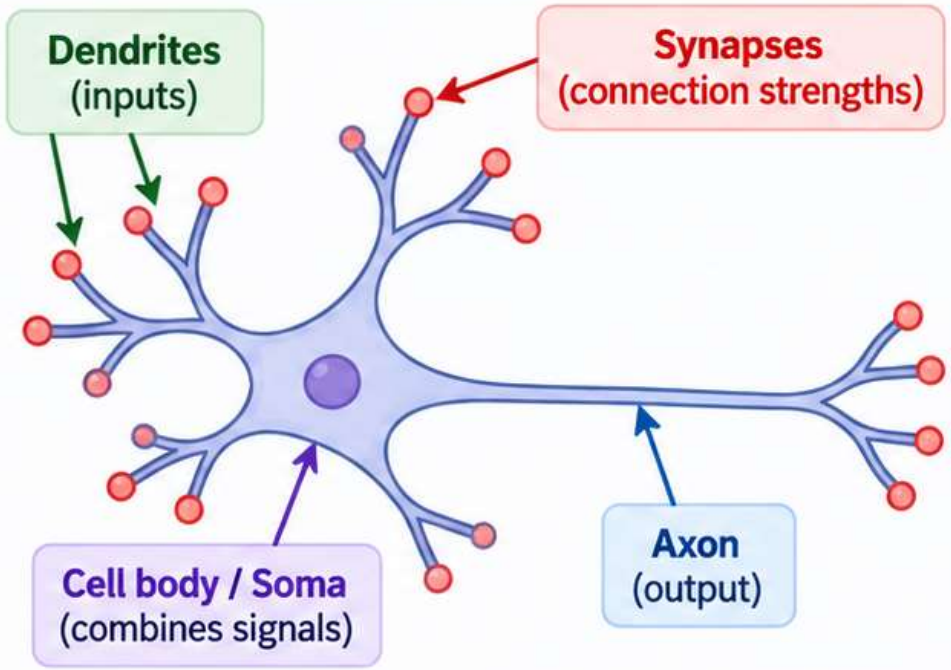
Inputs are combined.

Axon

An output is passed onward.

Biological inspiration for an artificial neuron

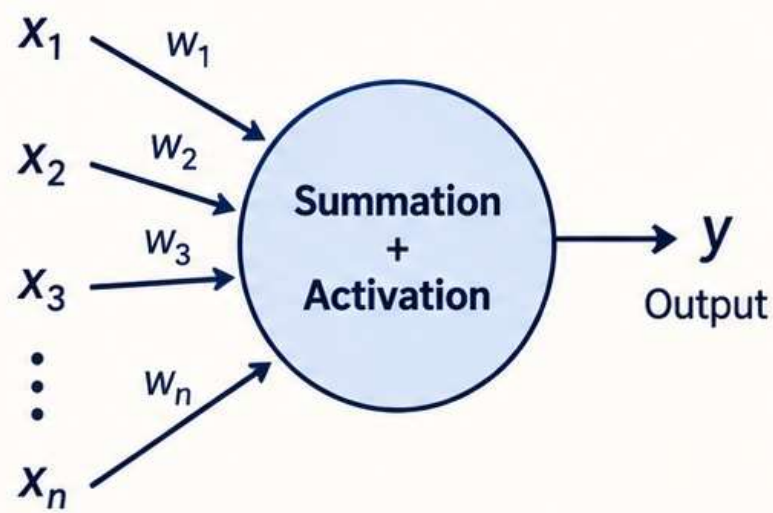
Biological neuron (real brain)



Mapping

Dendrites →	Inputs
Synapses →	Weights
Soma →	Summation / Activation
Axon →	Output

Artificial neuron (neural network)



1 Dendrites: receive inputs

Dendrites collect signals from other neurons.

2 Synapses: connection strengths

Each connection has a strength (weight).

3 Soma: combines inputs

The cell body adds up the inputs and decides whether to send a signal.

4 Axon: sends the output

The axon carries the output signal to other neurons.

From a biological neuron to an artificial neuron

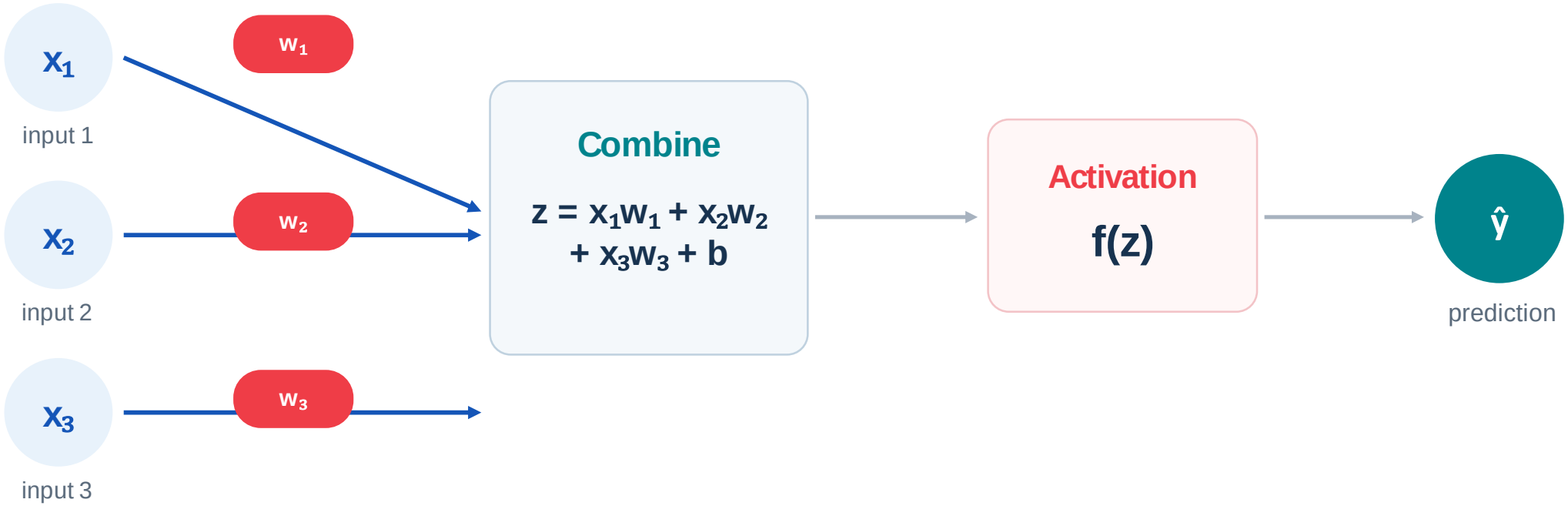
We copy the basic idea — not the full biology.

BIOLOGY	ARTIFICIAL NEURON	SYMBOL
Dendrites	Inputs	x_1, x_2, x_3
Synapses	Weights	w_1, w_2, w_3
Cell body	Combine inputs	$\Sigma(wx) + b$
Firing decision	Activation	$f(z)$
Axon	Output	$\hat{y}$

Easy memory trick: input → importance → combine → decide → output

The artificial neuron in one picture

Three inputs go in. One prediction comes out.



Nothing magical: multiply → add → make a decision.

What does a weight mean?

A weight tells the neuron how important an input is.

Large positive weight

+2.0

This input strongly pushes the answer UP.

Near zero weight

+0.05

This input has almost no effect.

Negative weight

−1.5

This input pushes the answer DOWN.

Think of weights like volume knobs: some signals are turned up, some down, some nearly muted.

What is the bias?

Bias is a small extra number that shifts the decision.

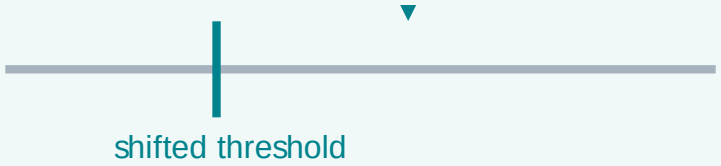
Without bias

The neuron may need a very specific input sum before it can activate.



With bias

The neuron can move the decision point left or right to fit the data better.

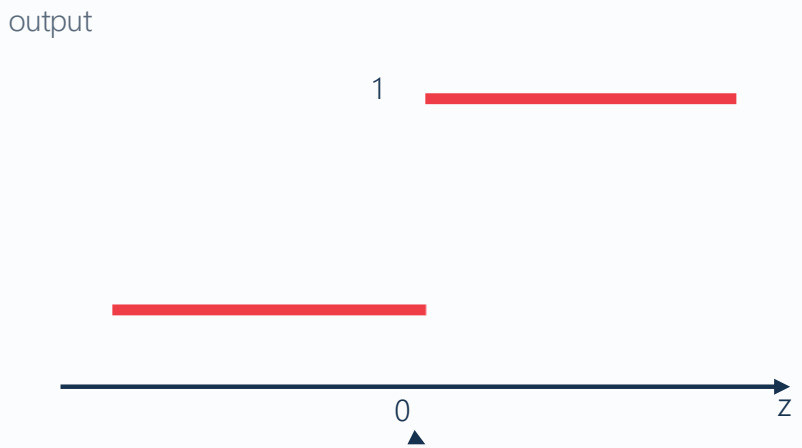


Easy memory trick: bias = threshold adjustment

Activation function: should the neuron “fire”?

For a perceptron, we often use a simple step decision.

Step activation



Simple rule

If z is large enough → output 1

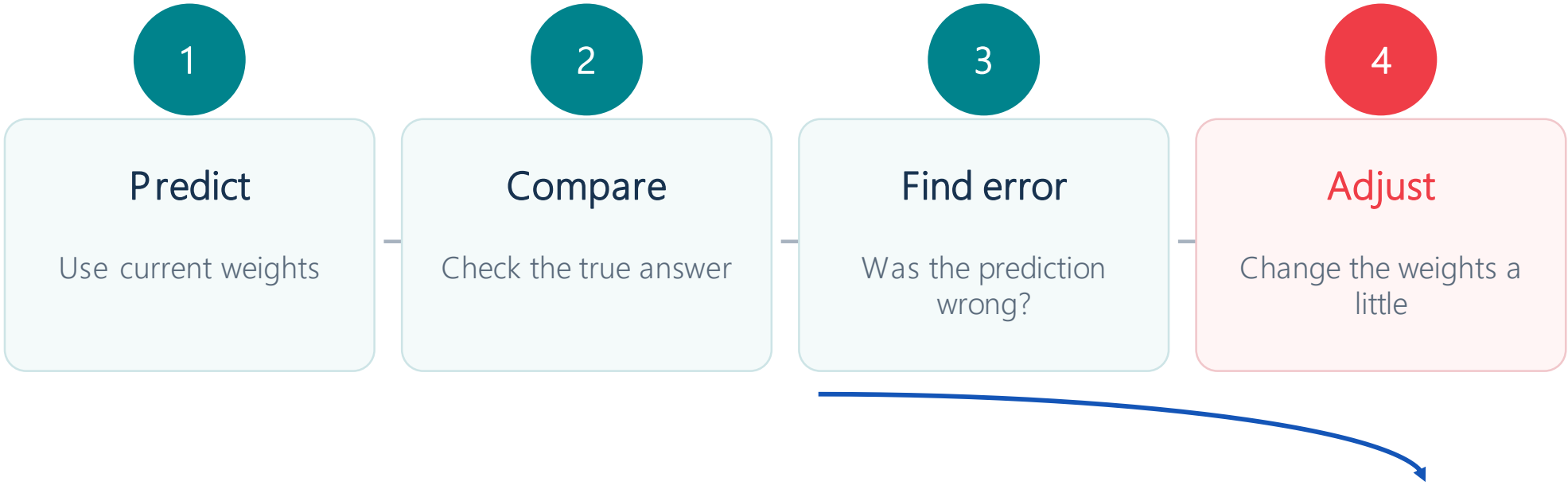
If z is too small → output 0

The activation converts the combined score into the final decision.

Perceptron = weighted sum + step decision

How does the neuron learn?

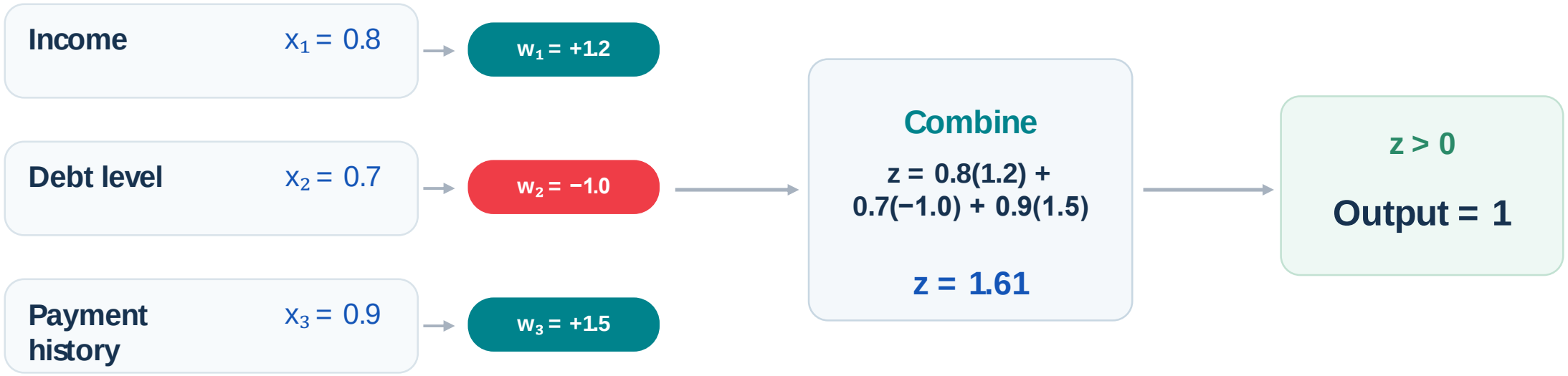
It improves by changing the weights after mistakes.



Learning = finding better weights so the error becomes smaller.

Tiny example: should we approve a loan?

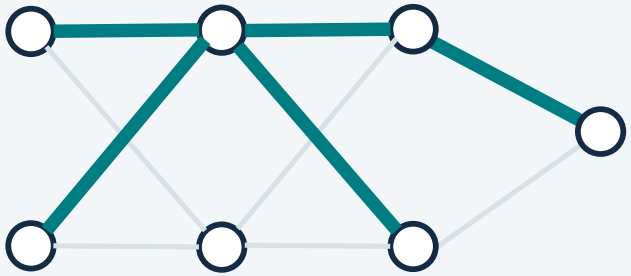
This is only to understand the neuron — real systems use more data and safeguards.



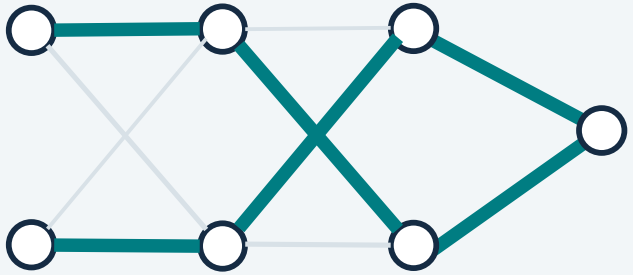
Main idea: inputs × weights → add → decide

Learning changes the connections

Before learning



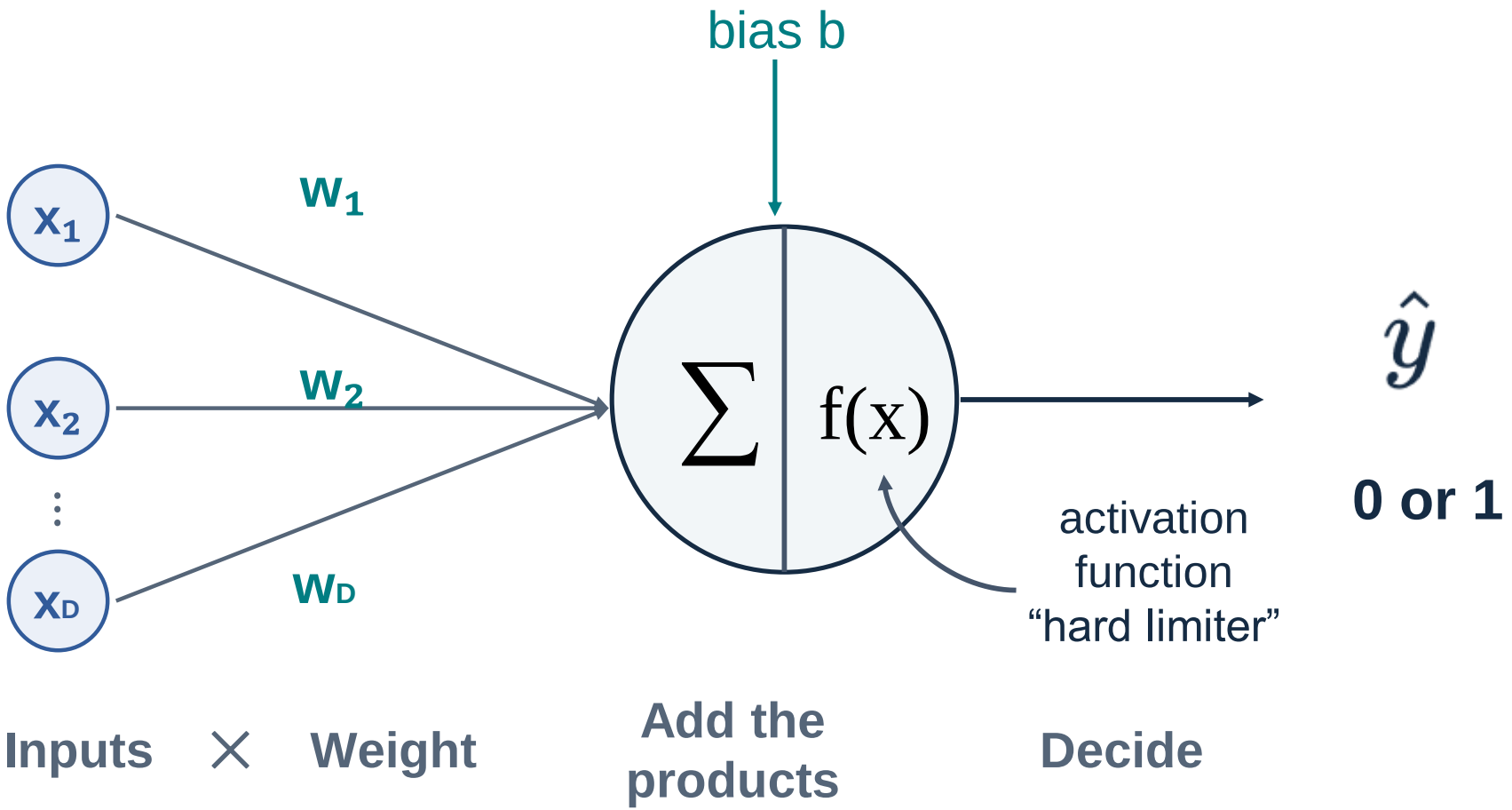
After learning



Same arrangement of connections; different strengths are emphasized.

Perceptron learning: adjust numerical weights and bias using labeled examples.

A perceptron in four operations



Give every symbol a job

GIVEN TO THE MODEL

$$\mathbf{x} \in \mathbb{R}^D$$

D **numerical** input features.

$$y \in \{0, 1\}$$

The correct label during training.

LEARNED PARAMETERS

$$\mathbf{w} \in \mathbb{R}^D$$

One weight for each feature.

$$b \in \mathbb{R}$$

One scalar bias.

COMPUTED OUTPUTS

$$s \in \mathbb{R}$$

The score before the threshold.

$$\hat{y} \in \{0, 1\}$$

The predicted class label.

Training supplies the true label y. Prediction produces the estimate ŷ.

Math pause: what does $w^T x$ calculate?

TWO-FEATURE CASE

$$\mathbf{w}^T\mathbf{x} = \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

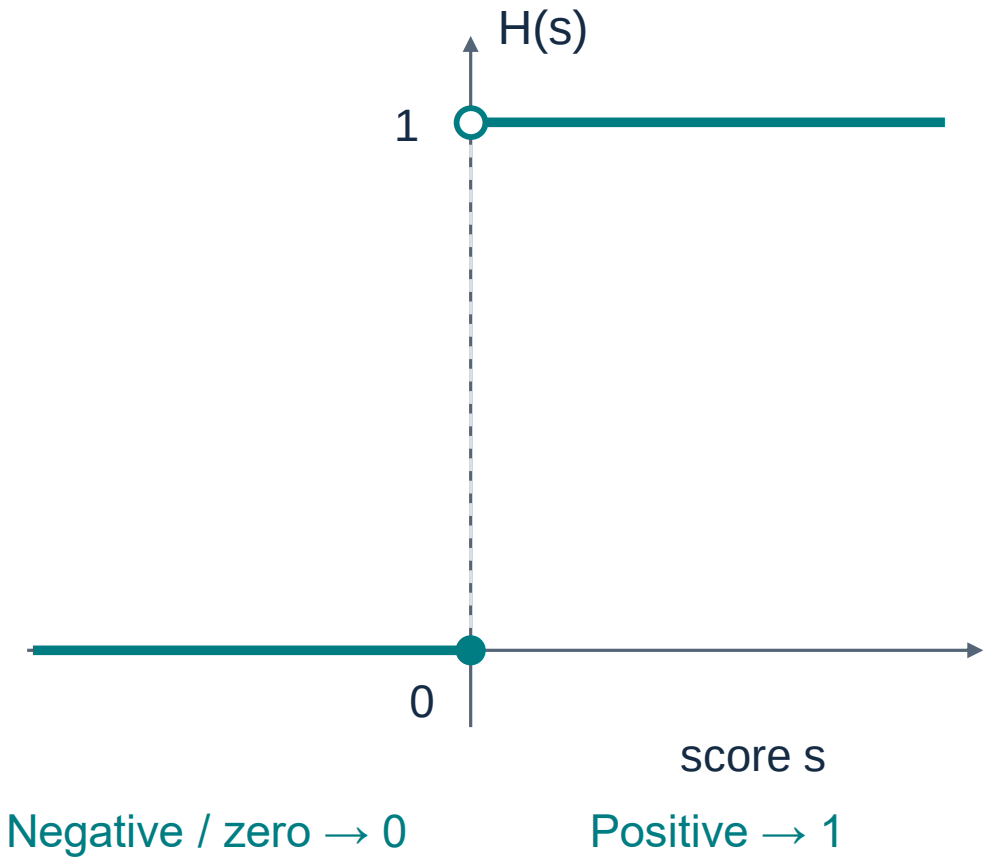
$$= w_1 x_1 + w_2 x_2$$

$$s = \underbrace{\mathbf{w}^T\mathbf{x}}_{\text{weighted sum}} + \underbrace{b}_{\text{offset}}$$

The “hard limiter” is a step function

$$\hat{y} = H(s) = \begin{cases} 1, & s > 0 \\ 0, & s \leq 0 \end{cases}$$

A hard class decision—not a probability.



Checkpoint 1 · What does a perceptron return?

PAUSE · DISCUSS · REVEAL

After the hard threshold, the output is…

A a probability such as 0.73.

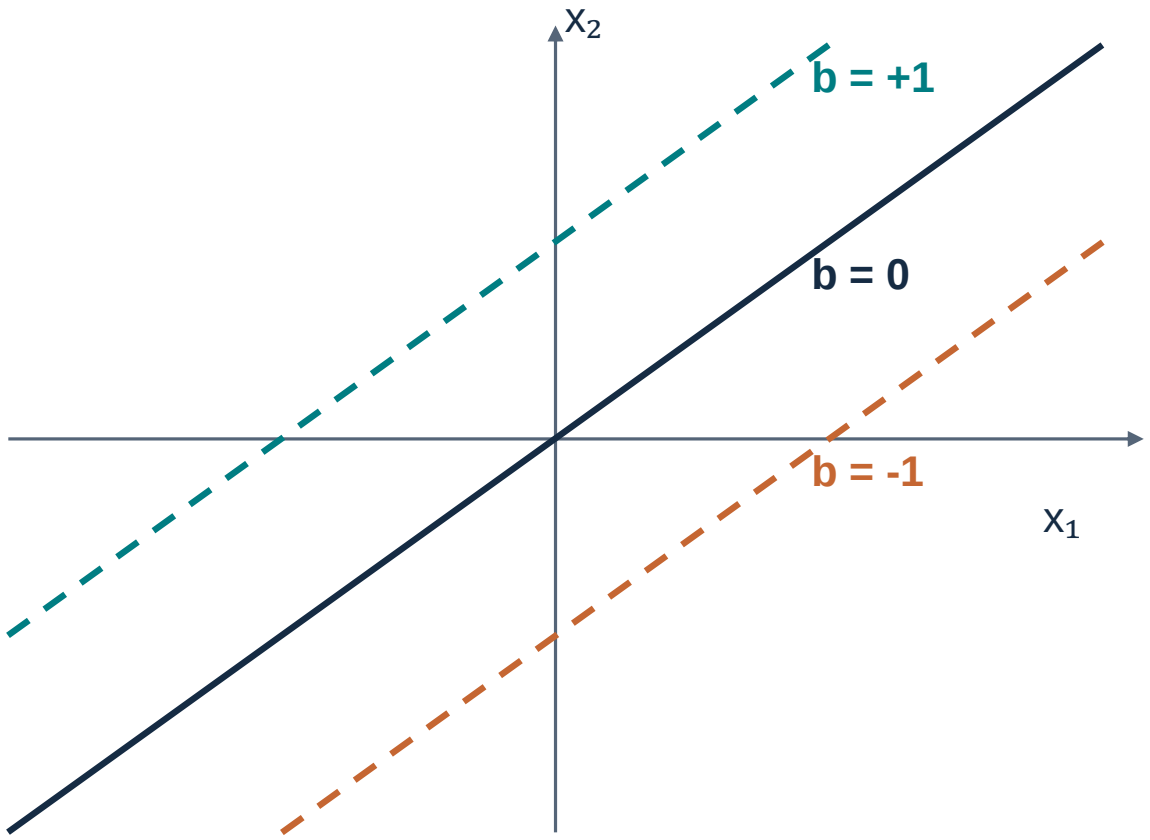
B a class label, either 0 or 1.

C the unbounded score $w^T x + b$.

D a continuously varying regression prediction.

B · The score is real-valued, but the hard threshold returns a class label.

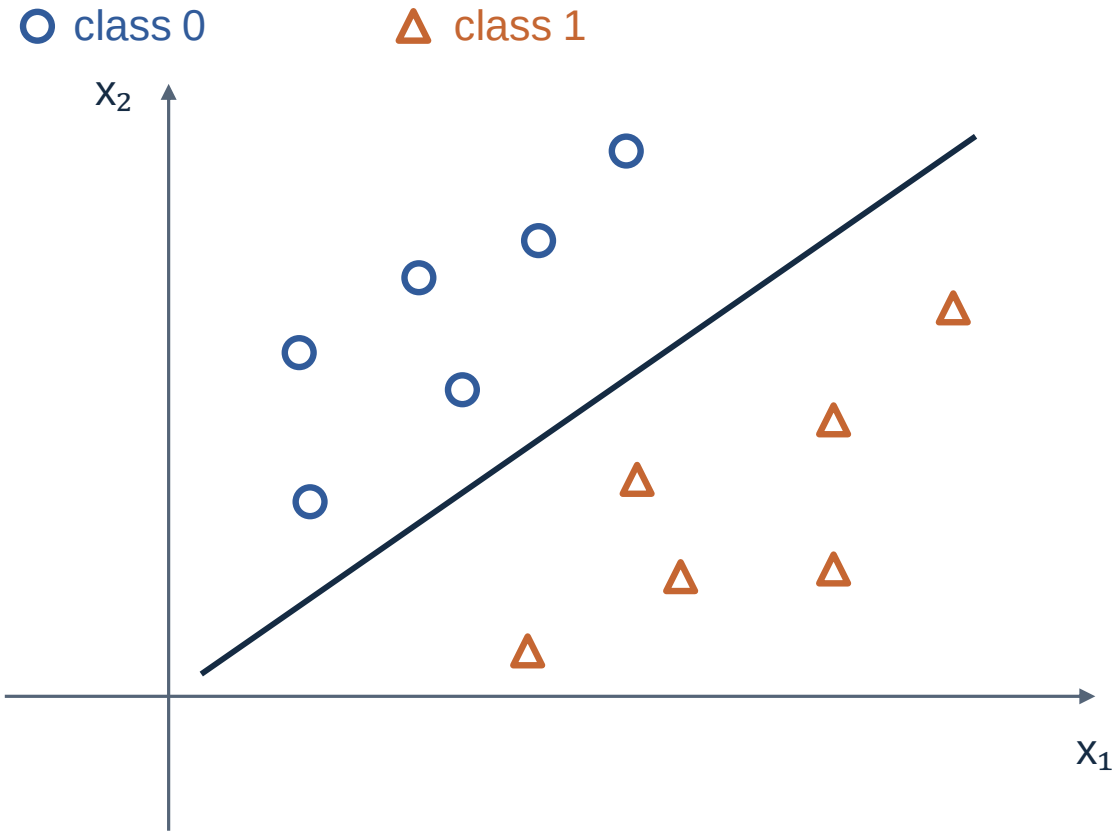
The bias lets the boundary move



HOLD THE WEIGHTS FIXED

Without a bias, the boundary must pass through the origin.

The boundary is where the score equals zero



TWO INPUT FEATURES

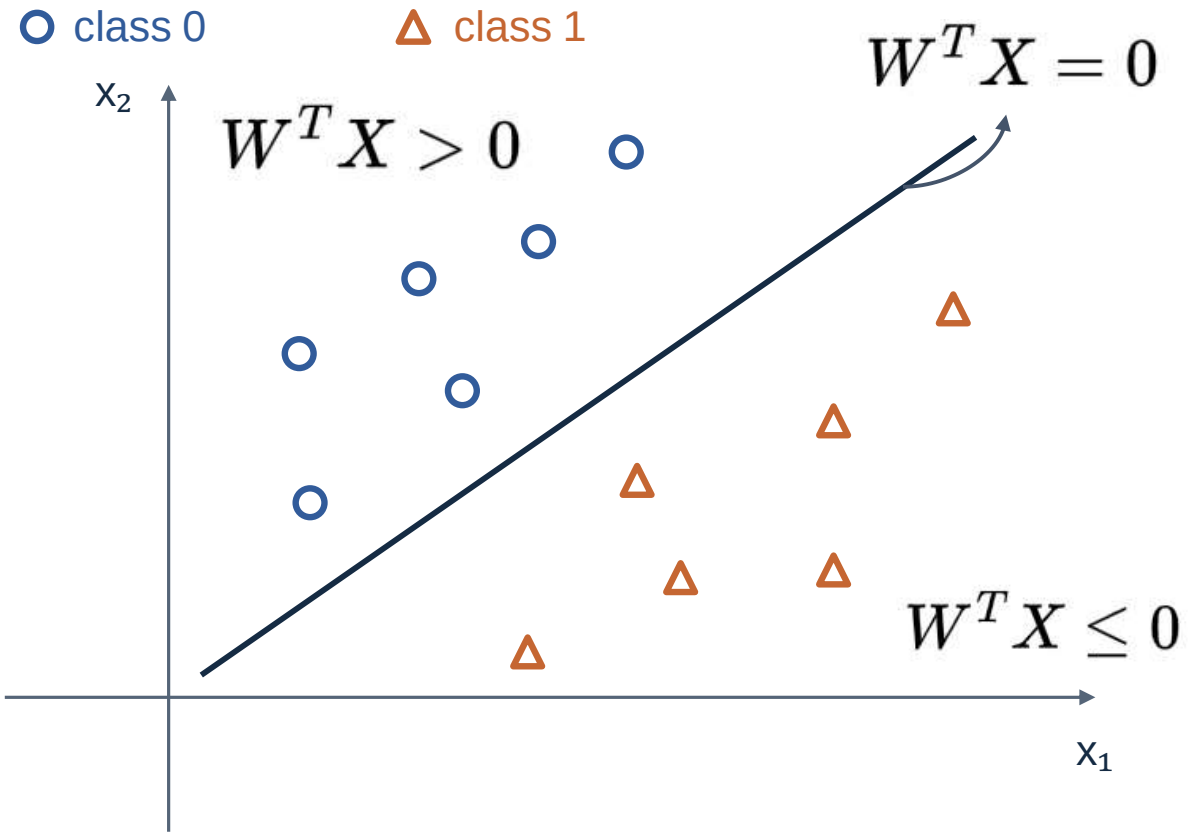
$$s = w_1 x_1 + w_2 x_2 + b$$

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$s > 0 \Rightarrow \hat{y} = 1$$

$$s \leq 0 \Rightarrow \hat{y} = 0$$

The boundary is where the score equals zero



ANY NUMBER OF INPUT FEATURES

$$x_0 = 1$$

$$w_0 = b$$

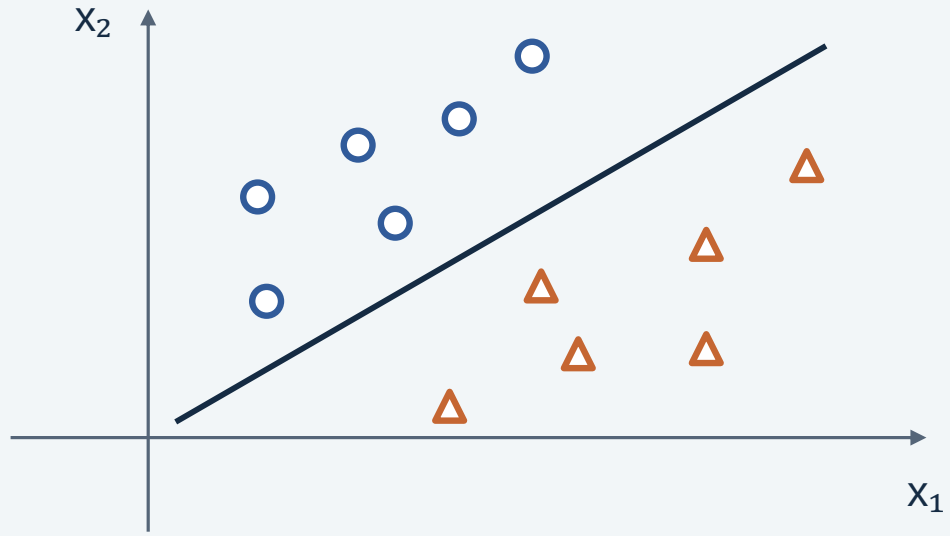
$$s = W^T X = \sum x_i w_i$$

A line is the two-feature case

Number of input features	Shape of the boundary
1 feature	A point / threshold on a number line
2 features	A line in a 2D feature space
3 features	A plane in a 3D feature space
D features	A hyperplane in a D-dimensional feature space

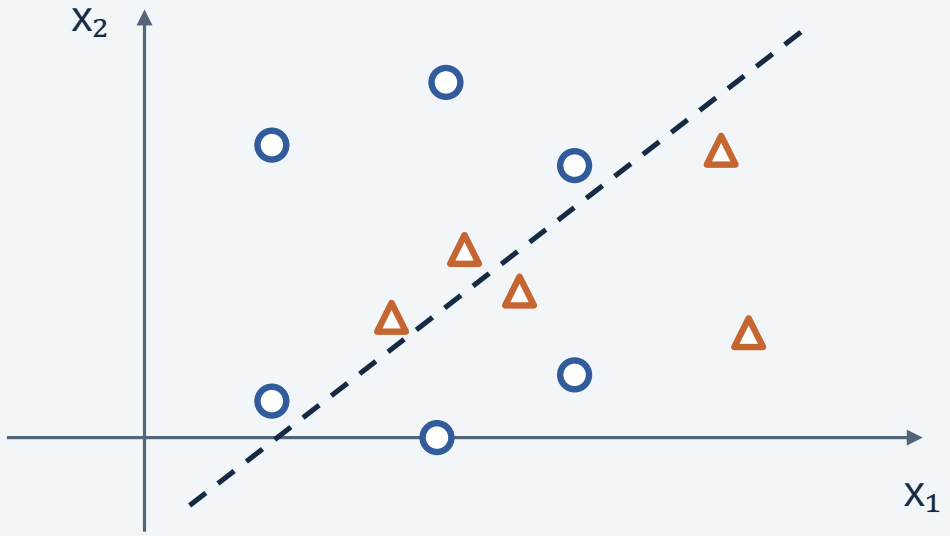
Can one boundary separate every example?

Linearly separable



At least one separating line exists.

Not linearly separable



Every candidate line leaves too many errors (underfitting).

Checkpoint 2 · What does changing b do?

PAUSE · DISCUSS · REVEAL

With a nonzero w held fixed, changing only the bias…

A changes the number of input features.

B translates the boundary without changing its orientation.

C automatically turns the line into a curved boundary.

D forces the boundary to pass through the origin.

B · The bias changes location; the weight direction determines orientation.

Prediction and learning are different operations

PREDICTION

Use the input

x arrives.

→

Calculate

Compute s and threshold it.

→

Return a label

Output ŷ.

LEARNING ADDS FEEDBACK

Compare ŷ with the correct label y.

Correct → keep the parameters W.
Wrong → update the parameters W.

Training: weight adjustment

Case 1 · The prediction is correct

True label y	Prediction ŷ	Error e = y − ŷ
0	0	0
1	1	0

$$\mathbf{w}_{t+1} = \mathbf{w}_t \qquad b_{t+1} = b_t$$

Correct classification → no weight update and no bias update.

Training: weight adjustment

Case 2 · A positive example was missed

TRUE 1 · PREDICTED 0

$$e = y - \hat{y} = 1 - 0 = +1$$

$$\boxed{\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \mathbf{x}_n}$$

$$\boxed{b_{t+1} = b_t + \eta}$$

η > 0

learning rate / step size

Add a scaled copy of the current input vector.

The score for this missed positive example moves upward.

Training: weight adjustment

Case 3 · A negative example was called positive

TRUE 0 · PREDICTED 1

$$e = y - \hat{y} = 0 - 1 = -1$$

Opposite mistake, opposite correction.

$$\boxed{\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{x}_n}$$

Subtract a scaled copy of the current input vector.

$$\boxed{b_{t+1} = b_t - \eta}$$

The score for this false positive example moves downward.

One rule covers all three cases

$$e = y_n - \hat{y}_n$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta e \mathbf{x}_n$$

$$b_{t+1} = b_t + \eta e$$

e	Action
0	No change
+1	Add ηx; add η to b
−1	Subtract ηx; subtract η from b

The prediction error decides the sign of the correction.

One correction is not necessarily a complete repair

Current example

Move its score in the useful direction.

→

Other examples

Their scores can change too.

→

Next visits

Check predictions again.

Learning is iterative.

A mistake can require more than one correction across repeated visits.

The training loop

```
choose initial w and b; choose η > 0
for each epoch up to max_epochs:
    mistakes = 0
    for each labeled example (x, y):
        s = wᵀx + b
        ŷ = 1 if s > 0 else 0
        e = y − ŷ
        if e ≠ 0:
            w = w + η e x
            b = b + η e
            mistakes = mistakes + 1
    if mistakes = 0: stop
```

One example at a time

Repeat full passes.

Stop after a mistake-free pass or the chosen limit.

Checkpoint 3 · Which correction is needed?

PAUSE · DISCUSS · REVEAL

For $y = 0$, $\hat{y} = 1$, and $\eta > 0$, which update is correct?

A $w \leftarrow w + \eta x$; $b \leftarrow b + \eta$

B $w \leftarrow w - \eta x$; $b \leftarrow b - \eta$

C w stays fixed; only b changes

D No change, because the prediction was positive

B · Here $e = y - \hat{y} = -1$, so both the vector and bias corrections are subtractive.

Worked example · Two records

Features: petal length x_1 and petal width x_2, measured in cm.

Example / source row	Species	x_1	x_2	Binary label y
A / row 0	Setosa	1.4	0.2	0
B / row 50	Versicolor	4.7	1.4	1

Binary task: Setosa = 0; not Setosa = 1.

CHOSEN INITIALIZATION

$$\mathbf{w} = (0, 0), \quad b = 0$$

CHOSEN TRAINING PROCEDURE

$$\eta = 1, \quad A \rightarrow B \text{ each epoch}$$

Epoch 1 · Start with example A

A: x = (1.4, 0.2), y = 0

CURRENT PARAMETERS

$$\mathbf{w} = (0, 0), \quad b = 0$$

$$s = (0)(1.4) + (0)(0.2) + 0 = 0$$

$$\hat{y} = H(0) = 0$$

$$e = 0 - 0 = 0$$

PARAMETERS AFTER A

$$\mathbf{w} = (0, 0), \quad b = 0$$

Correct → no change.

Epoch 1 · Example B triggers an update

B: x = (4.7, 1.4), y = 1

$$s = (0)(4.7) + (0)(1.4) + 0 = 0$$

$$\hat{y} = 0, \qquad e = 1 - 0 = 1$$

$$\mathbf{w}_{\text{new}} = (0, 0) + (1)(1)(4.7, 1.4) = (4.7, 1.4)$$

$$b_{\text{new}} = 0 + (1)(1) = 1$$

Epoch 2 · Revisit A with the learned parameters

A: x = (1.4, 0.2), y = 0

$$s = (4.7)(1.4) + (1.4)(0.2) + 1$$
$$= 6.58 + 0.28 + 1 = 7.86$$

What is ŷ? What is e? Do we add or subtract?

ŷ = 1; e = 0 − 1 = −1. Subtract the input vector and subtract 1 from b.

The correction helps—but A is still wrong

$$\mathbf{w}_{\text{new}} = (4.7, 1.4) - (1.4, 0.2) = (3.3, 1.2)$$

$$b_{\text{new}} = 1 - 1 = 0$$

Check the same input again (example B is correct)

$$s_A = (3.3)(1.4) + (1.2)(0.2) + 0$$

$$= 4.86 > 0 \quad \Rightarrow \quad \hat{y}_A = 1$$

7.86 → 4.86

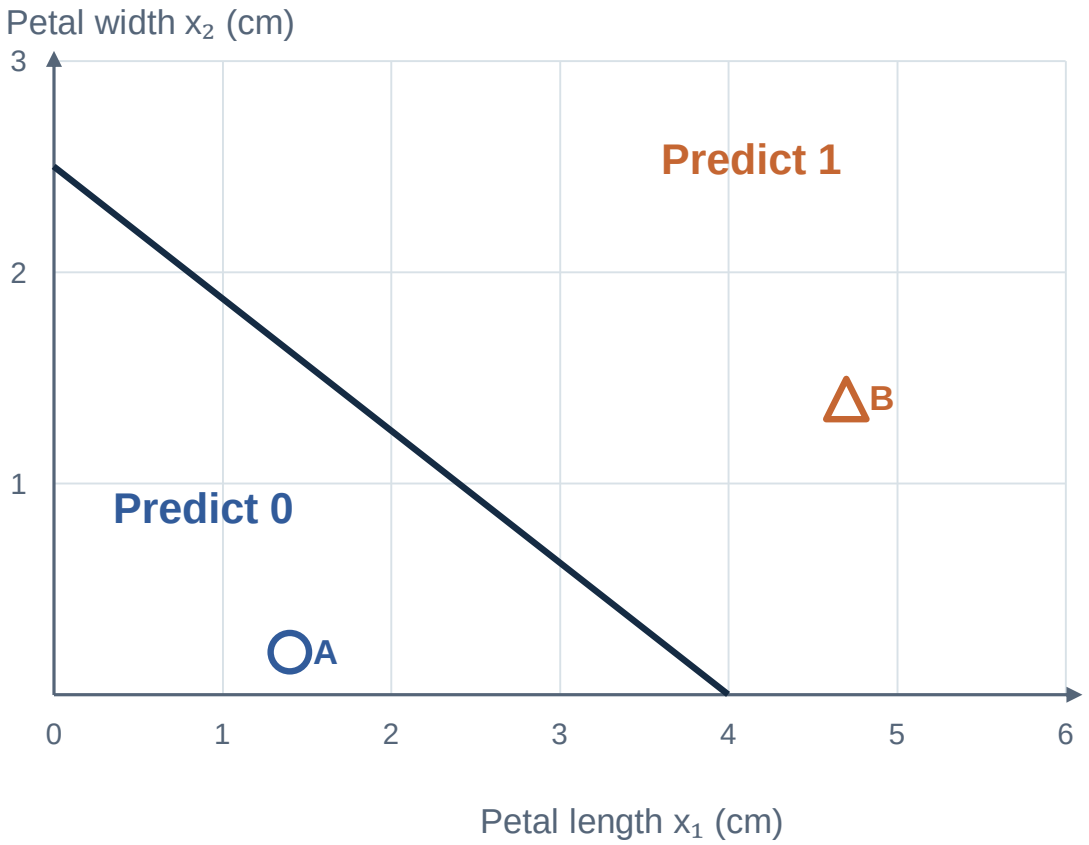
The score moved downward, but it did not cross zero.

Keep the new parameters and repeat the passes

Completed epoch	w_1	w_2	b	Mistakes during pass
1	4.7	1.4	1	1
2	3.3	1.2	0	1
3	1.9	1.0	−1	1
4	0.5	0.8	−2	1
5	0.5	0.8	−2	0 → stop

Epoch 4 finds the separator. Epoch 5 confirms a complete mistake-free pass.

Verify the final classifier



$$s = 0.5x_1 + 0.8x_2 - 2$$

$$s_A = 0.5(1.4) + 0.8(0.2) - 2 = -1.14$$

A → class 0: correct

$$s_B = 0.5(4.7) + 0.8(1.4) - 2 = 1.47$$

B → class 1: correct

Decision boundary: $0.5x_1 + 0.8x_2 - 2 = 0$.

Your turn · Predict one more record

PREDICTION ONLY · KEEP w AND b FIXED

$$\mathbf{x}_C = (5.1, 1.8)$$

$$s_C = 0.5(5.1) + 0.8(1.8) - 2$$

What class will the model return?

sC = 2.55 + 1.44 − 2 = 1.99 > 0 → class 1 (“not Setosa”).

Checkpoint 4 · What can we conclude?

PAUSE · DISCUSS · REVEAL

Which statement about this training procedure is correct?

A. Every example causes a weight update.

B. Setting $\eta = 1$ guarantees success on any dataset.

C. A full pass with zero mistakes satisfies our convergence check.

D. Reaching max_epochs proves the data cannot be separated.

C · A mistake-free pass is the check. A run limit alone proves neither convergence nor nonseparability.

What does the learning rate control?

$$\Delta \mathbf{w} = \eta e \mathbf{x}, \qquad \Delta b = \eta e$$

e

Which direction?

The kind of mistake determines the sign.

η

How much?

A positive factor scales that correction.

η = 1

Our walkthrough

Keeps the arithmetic transparent.

For this lecture: use a fixed η > 0. η = 0 would prevent learning.

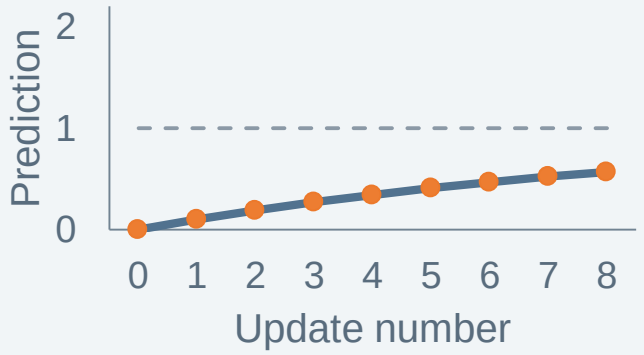
Learning rate: too low, suitable, or too high?

Example: a numerical prediction starts at 0 and learns toward target 1.

New prediction = current prediction + η × (target − current prediction)

Too low

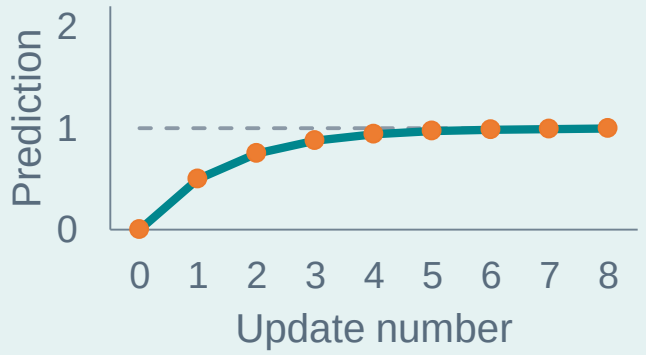
η = 0.1



Small steps need many updates

Suitable

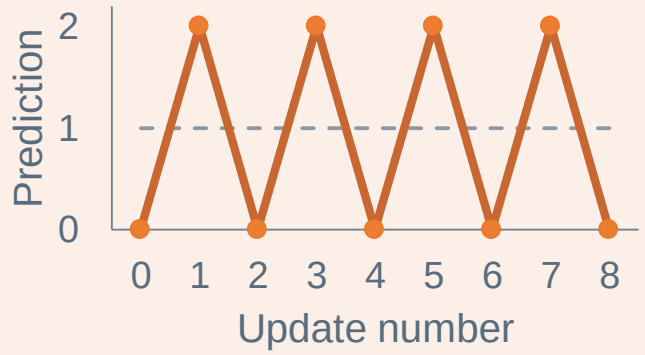
η = 0.5



Approaches the target faster

Too high

η = 2.0



Overshoots and keeps bouncing

Dashed line: target = 1. Rates shown apply to this example.

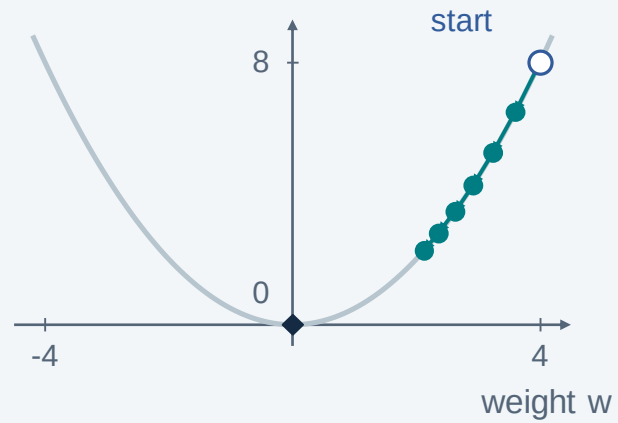
The learning rate controls the size of each correction.

Appendix A4 · Learning rate: how big is each move?

Gradient descent uses the slope of the loss. The learning rate η scales each move.

Too small

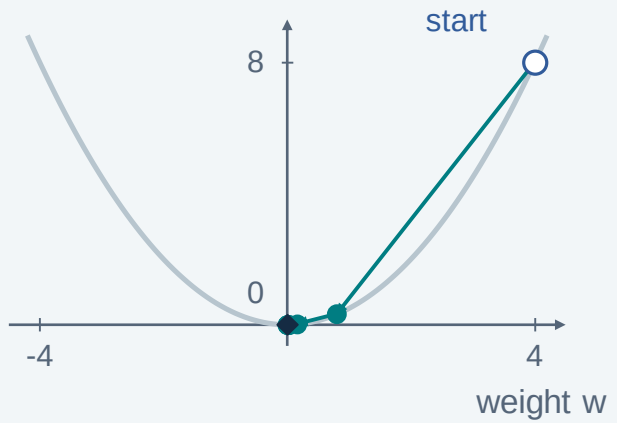
η = 0.1 in this example



Small moves.
Slow progress.

Well chosen

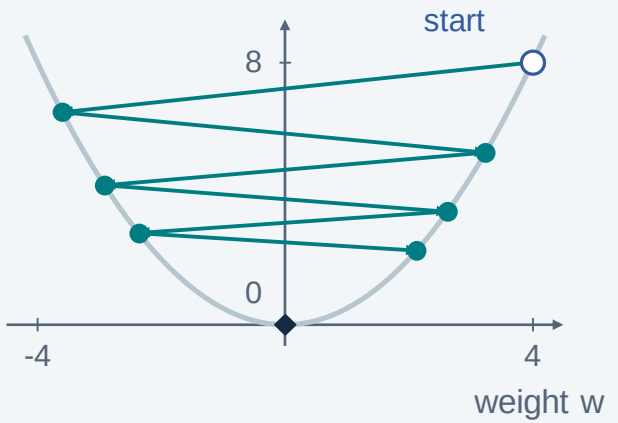
η = 0.8 in this example



Near the minimum
in a few moves.

Large: overshoots

η = 1.9 in this example



Back-and-forth jumps.
Larger rates can diverge.

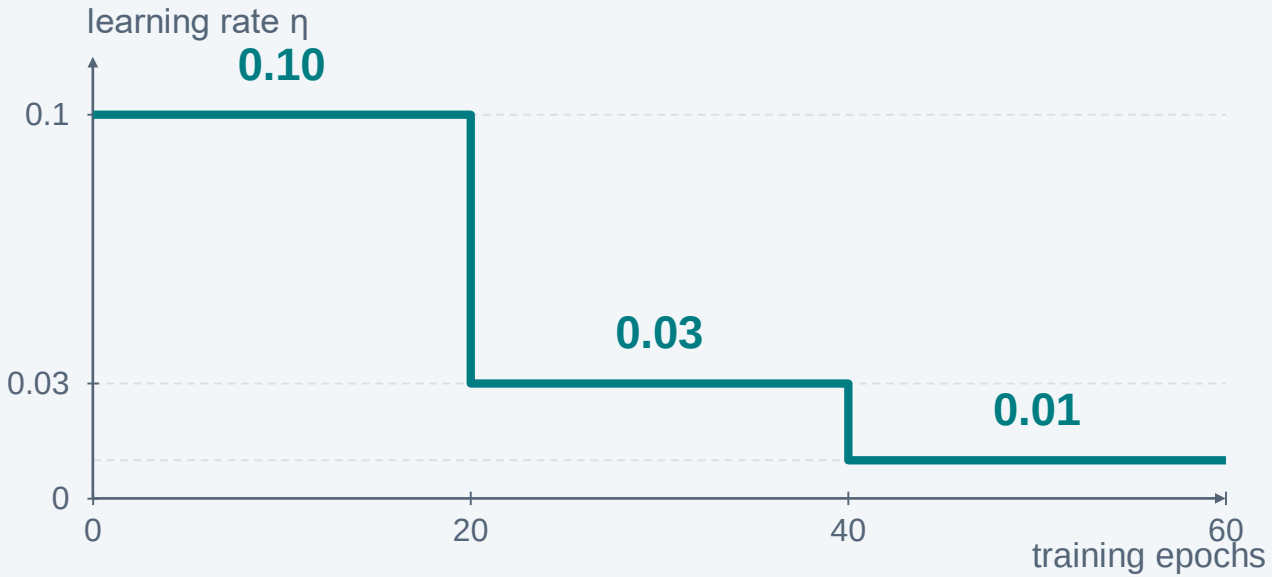
Same loss curve and starting point; six updates are shown. The best rate depends on the problem.

These graphs show gradient descent not the perceptron's mistake-only update rule.

Appendix A4 · Learning-rate decay: progress, then fine tuning

Reduce η as training progresses, rather than keeping it fixed throughout.

An example step-decay schedule



Illustrative values—not universal default settings.

EARLIER

Larger, stable steps make faster progress.

LATER

Smaller steps help fine-tune and reduce bouncing.

WHY NOT SMALL FROM THE START?

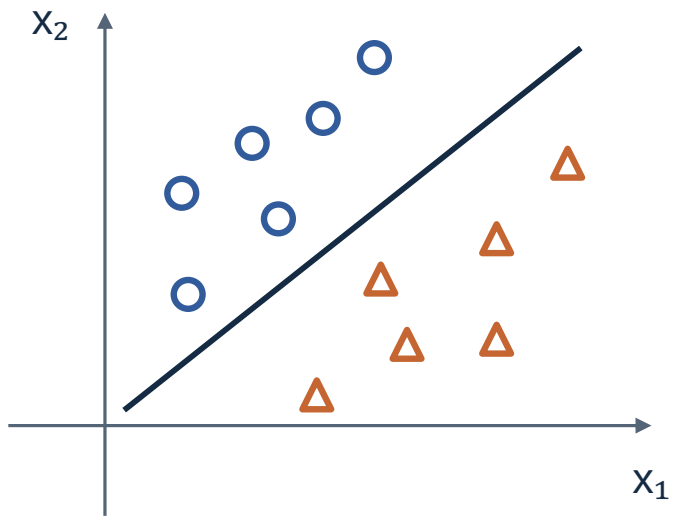
Learning could be slow throughout training.

Decay can help, but reducing the learning rate too early can slow learning.

When does the perceptron converge?

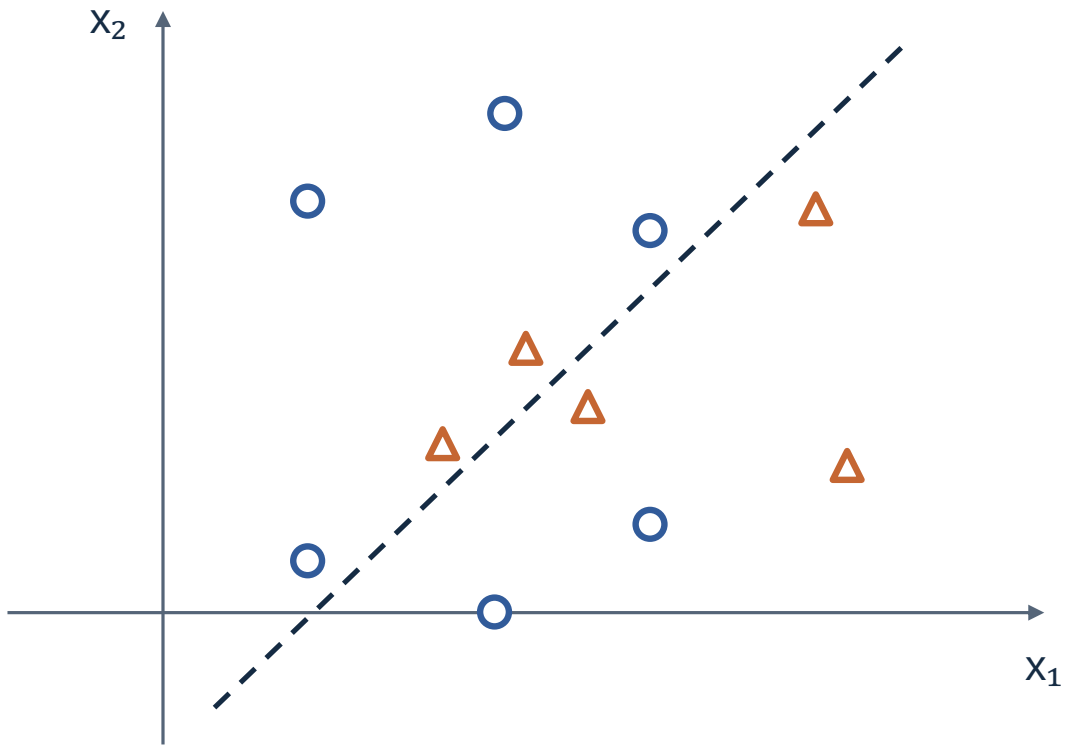
The convergence guarantee requires a linearly separable training set.

Use a fixed positive learning rate.
Keep revisiting the finite training set.



Then learning eventually reaches a complete pass with no training mistakes.

When no separating line exists, more passes are not enough



A capacity limit

Changing weights moves a hyperplane.
It does not turn that hyperplane into a curve.

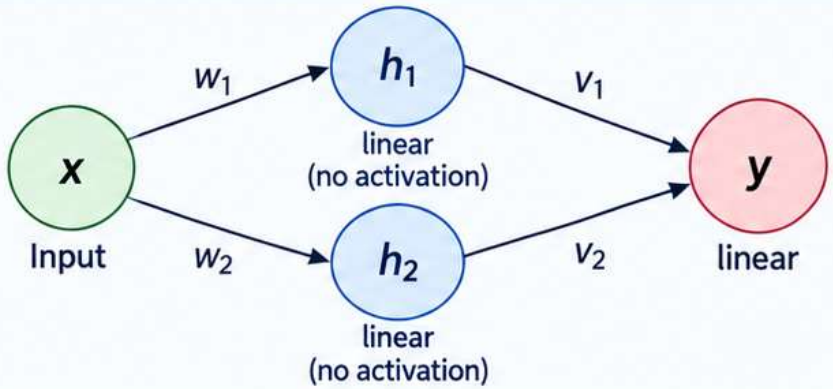
A maximum-epoch limit can stop the run without solving the classification problem.

Stopping because of a limit is not the same as convergence.

Why do we need activation functions?

Without activation

All neurons are linear (no activation function).



The whole network simplifies to one linear neuron:

$$\begin{aligned} y &= v_1(w_1x + b_1) + v_2(w_2x + b_2) + b \\ &= (v_1w_1 + v_2w_2)x + (v_1b_1 + v_2b_2 + b) \\ &= Wx + B \end{aligned}$$

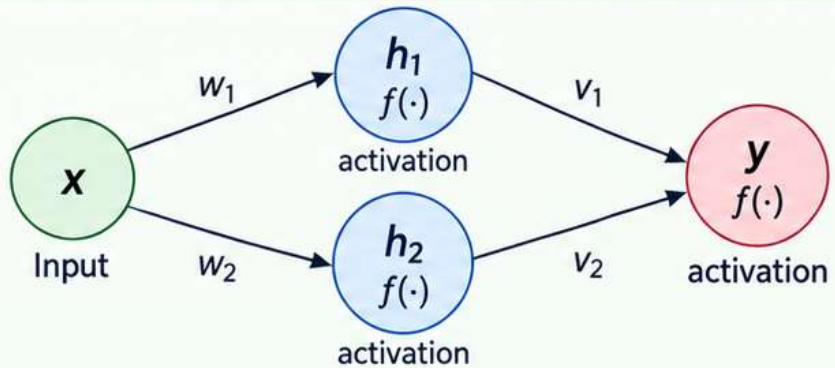
$y = Wx + B$

Single linear neuron

Even with more linear layers, it is still just one linear model.

With activation

Neurons use a nonlinear activation function (e.g., ReLU, sigmoid).



Now the model becomes nonlinear.

$$y = v_1 f(w_1x + b_1) + v_2 f(w_2x + b_2) + b$$

This cannot be reduced to one linear layer.

Because $f(\cdot)$ is a nonlinear function, the network can learn more complex relationships.

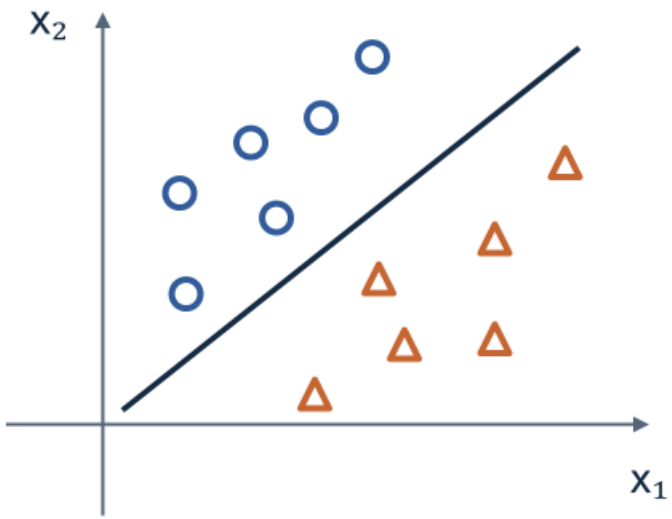
Summary:

1. **Without activation:** many layers collapse into one linear layer.
2. **With activation:** the network can learn complex nonlinear patterns.

When does the perceptron converge?

The convergence guarantee requires a linearly separable training set.

Use a fixed positive learning rate.
Keep revisiting the finite training set.



Then learning eventually reaches a complete pass with no training mistakes.

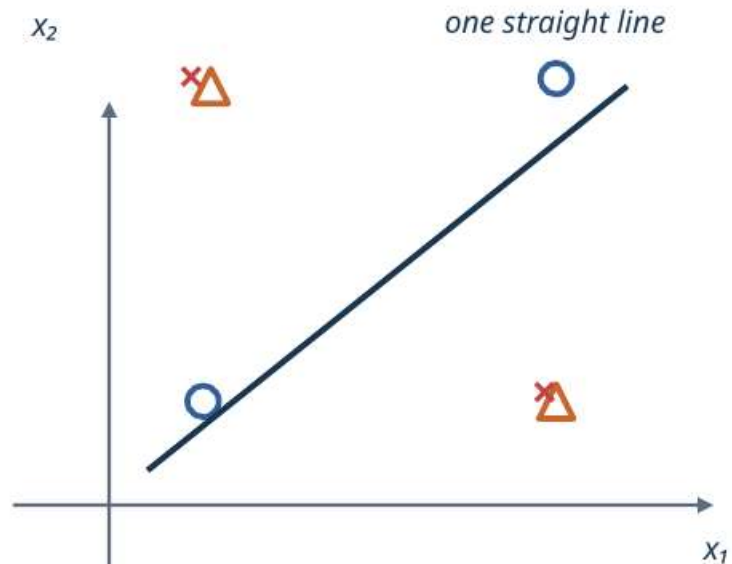
What if one line is not enough?

A single perceptron can draw only one straight decision boundary.

That works only when the classes are linearly separable.

If no straight line can separate the classes, there is no zero-error solution for one perceptron.

Example: XOR is not linearly separable.



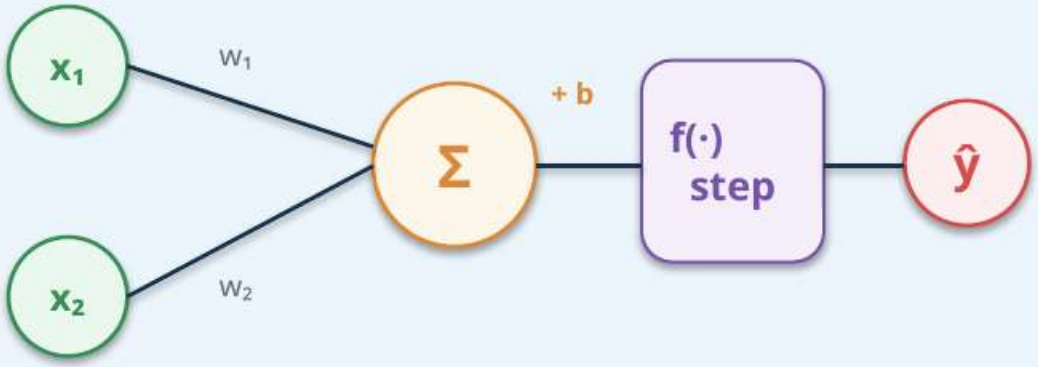
No matter where we move the line, at least one class is mixed with the other.

No separating line → the perceptron cannot converge to zero training mistakes.

1. Start with one perceptron

A perceptron can classify points only when one straight decision boundary is enough.

One perceptron

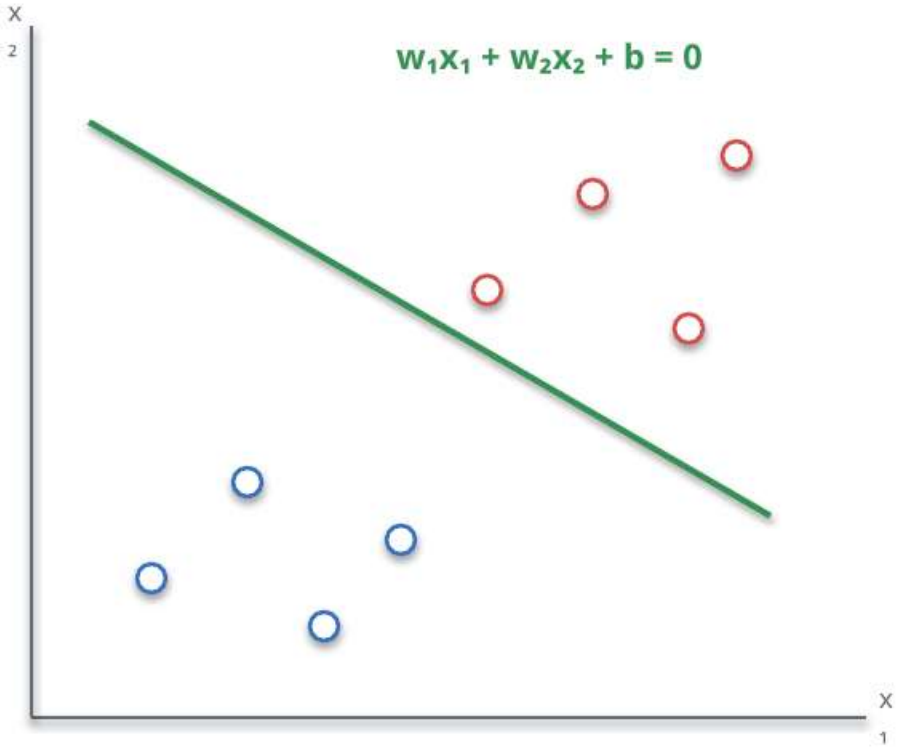


$$z = w_1x_1 + w_2x_2 + b$$

$$\hat{y} = f(z)$$

The step activation decides which side of the boundary is class 0 or class 1.

Its decision boundary is still a line



This works when the classes are linearly separable.

A single perceptron already has an activation function, but it can create only one linear boundary.

2. The problem: nonlinear separation

XOR is the simplest example where one perceptron is not enough.

XOR truth table

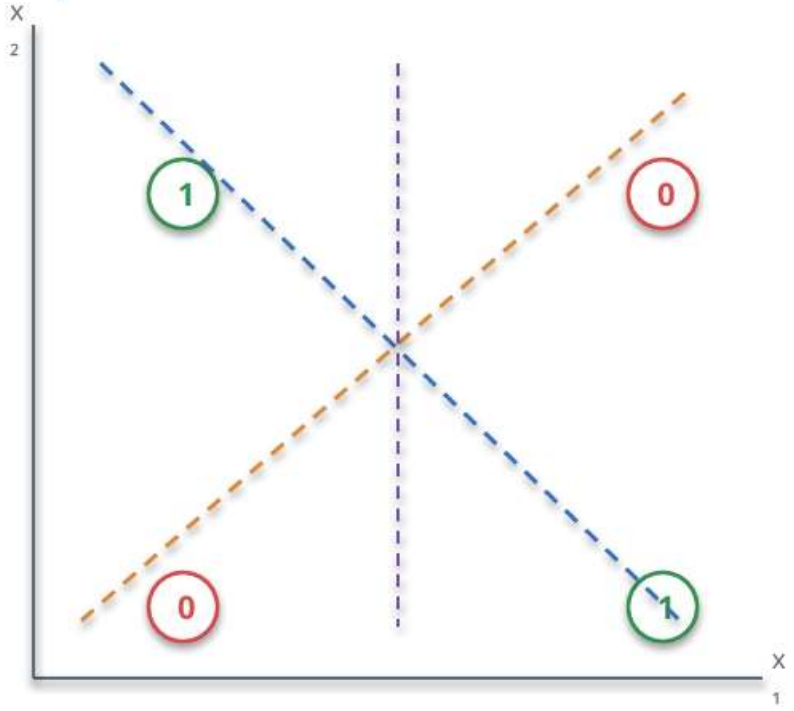
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

The 1s are on opposite corners.

The 0s are on the other opposite corners.

No single line can separate them.

XOR points



Try any line → at least one point is classified incorrectly.

Why one perceptron fails

It computes one score:

$$z = w_1x_1 + w_2x_2 + b$$

Then the activation assigns one side of one line to class 1 and the other side to class 0.

XOR needs multiple boundaries.

So we need multiple neurons — not just one perceptron.

Important: the limitation is not "no activation." The limitation is one linear decision boundary.

Why does XOR fail for one perceptron?

XOR puts opposite corners into the same class.

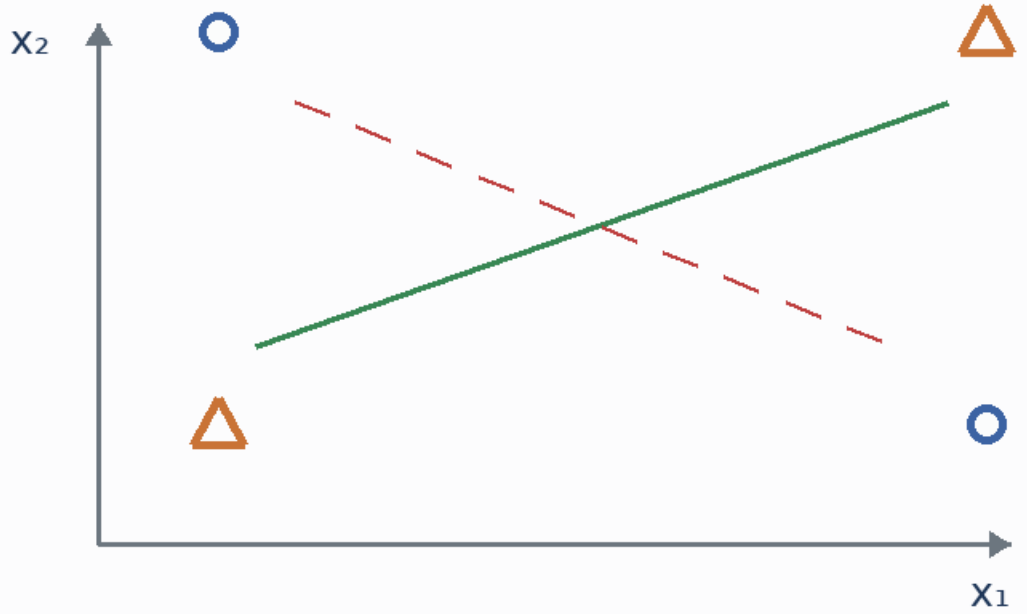
A single perceptron creates only one straight decision boundary.

In XOR, the classes alternate across the square: the two blue points are on one diagonal and the two orange points are on the other diagonal.

Therefore, any single line will split at

One perceptron: $y = step(w_1x_1 + w_2x_2 + b)$
It can draw one line only.

XOR points in the input space



No single line can separate the two XOR classes.

Key idea: **XOR is not linearly separable, so one perceptron is not enough.**

So how do we go beyond a single perceptron?

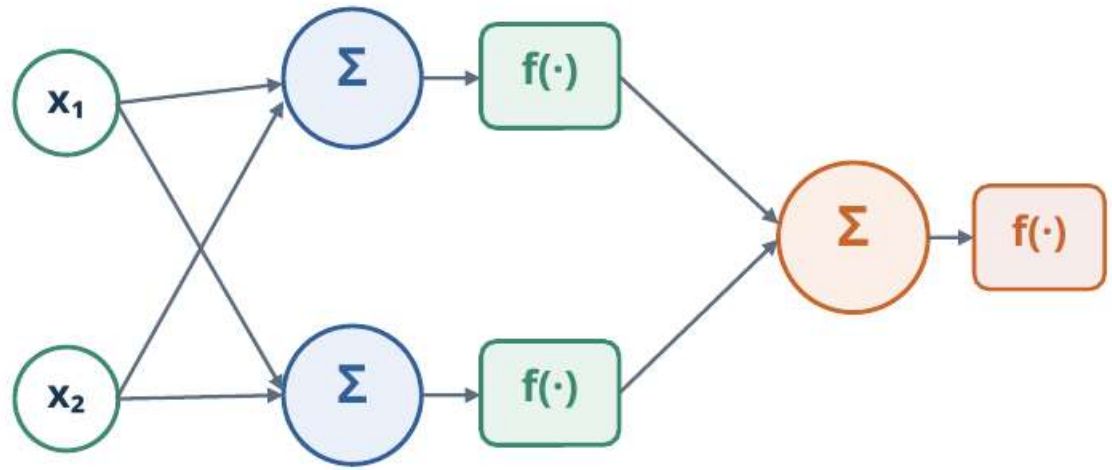
1 Use multiple neurons

Each neuron can learn a different linear boundary.

2 Add nonlinear activation

The activation lets the next layer combine those boundaries in a nonlinear way.

Many linear pieces + nonlinear activation → nonlinear decision boundary



Result: a nonlinear region can be built from several learned boundaries.



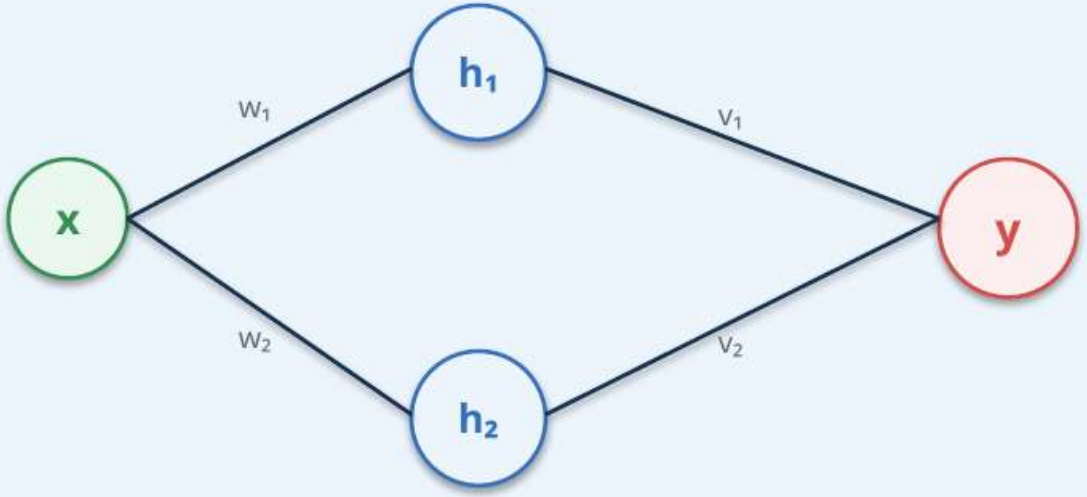
piecewise / nonlinear boundary

A single perceptron is limited. A multilayer network with nonlinear activations can model nonlinearly separable data.

3. But adding layers without nonlinear activation does not help

Several linear layers collapse into one equivalent linear layer.

One input → 2 hidden neurons → one output



$h_1 = w_1x + b_1$ $\quad$ $h_2 = w_2x + b_2$

$$y = v_1h_1 + v_2h_2 + b_3$$

No f(·) between layers

Substitute the hidden neurons

$$y = v_1(w_1x + b_1) + v_2(w_2x + b_2) + b_3$$

$$=$$

$$(v_1w_1 + v_2w_2)x + (v_1b_1 + v_2b_2 + b_3)$$

$$y = Wx + B$$

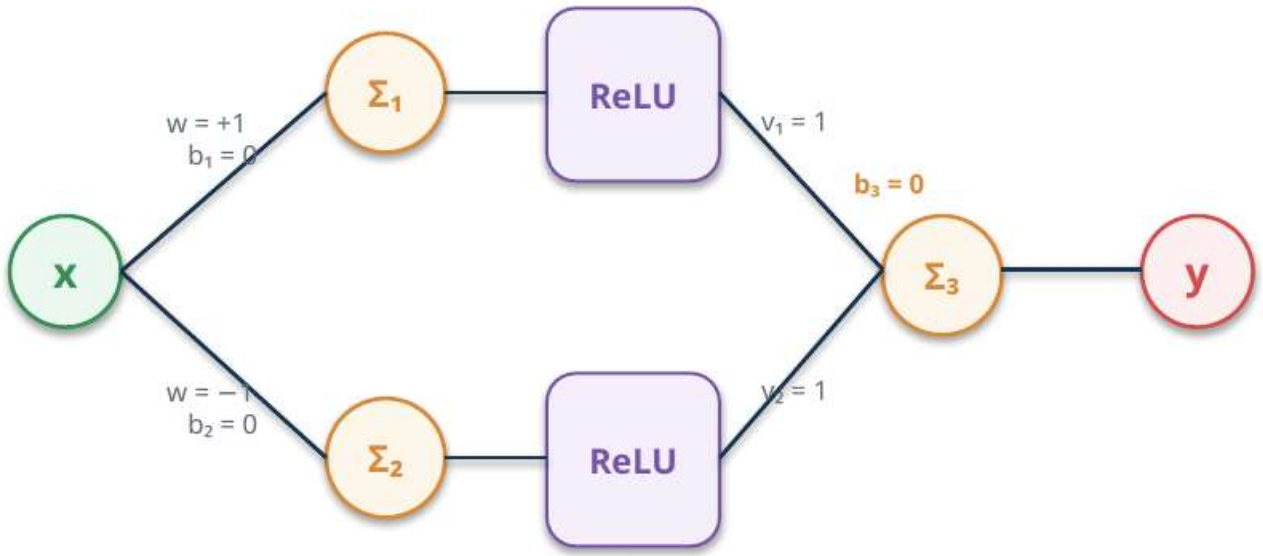
The entire network is still one linear model.

10, 100, or 1000 linear layers still cannot create a nonlinear decision boundary.

4. Nonlinear activation changes what the network can represent

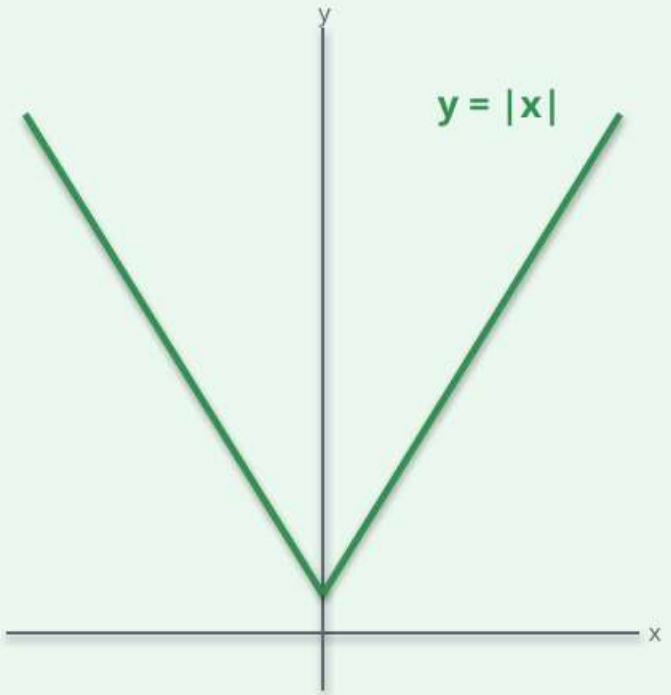
A tiny numerical example: two ReLU units can create a V-shaped function that one linear neuron cannot.

f(·) is separate from each neuron



$h_1 = \mathrm{ReLU}(x)$ $h_2 = \mathrm{ReLU}(-x)$ $y = h_1 + h_2 = |x|$

See the nonlinearity



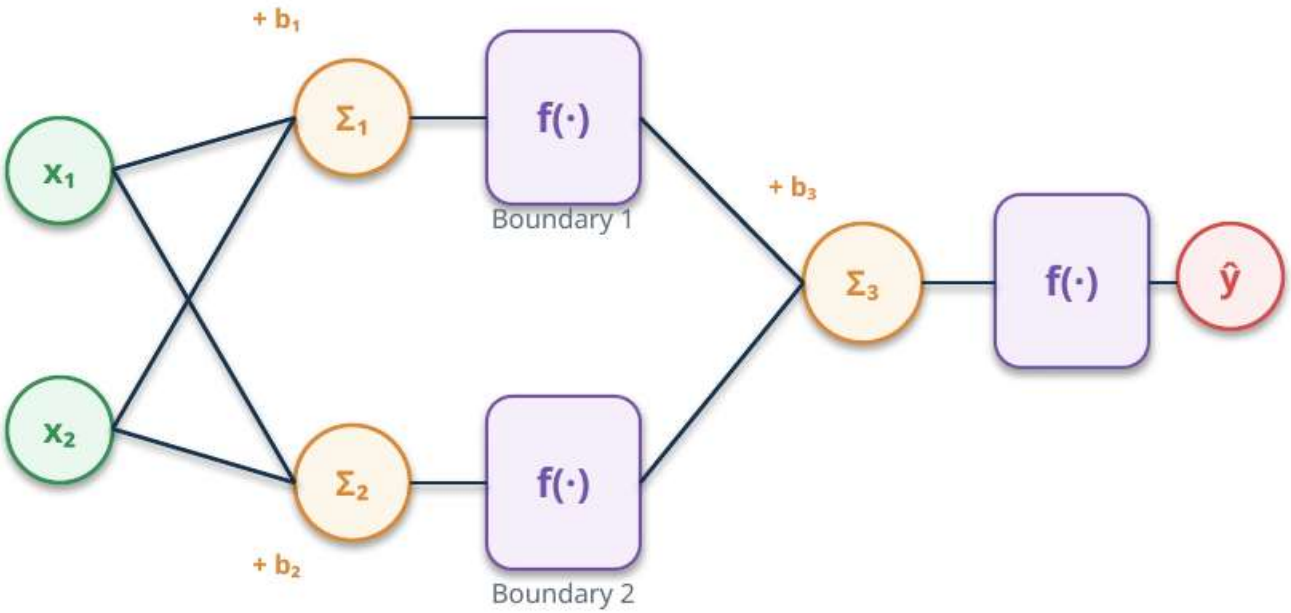
This cannot be written as one equation $y = Wx + B$ for all x.

The activation breaks the "all layers collapse into one linear layer" simplification.

5. Putting it together: why activation helps with XOR

Hidden neurons create intermediate decisions; nonlinear activations let the next layer combine them into a nonlinear region.

Two hidden neurons can learn two different boundaries



Nonlinear activations make the hidden outputs useful as new features for the final neuron.

The message for students

1 perceptron → 1 straight boundary

More layers, no nonlinear f(·) → still linear

Multiple neurons + nonlinear f(·) → nonlinear regions

That is why activation functions such as ReLU, sigmoid, tanh, or step are placed between layers.

XOR is the classic demonstration.

Takeaway: activation gives a multilayer network the nonlinearity needed to model patterns a single linear boundary cannot represent.

Why does AND succeed with one perceptron?

AND is linearly separable.

Only one point should be class 1:

(1,1) → 1

All other points → 0

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

One straight line can isolate (1,1).

A single boundary is enough



Perceptron: $y = \text{step}(x_1 + x_2 - 1.5)$

AND: one set of weights solves all four cases

Choose simple parameters

$w_1 = 1$, $w_2 = 1$, $b = -1.5$

$z = x_1 + x_2 - 1.5$

$y = step(z)$

(0,0)	z = −1.5	→ **0**
(0,1)	z = −0.5	→ **0**
(1,0)	z = −0.5	→ **0**
(1,1)	z = +0.5	→ **1**

What the perceptron learns



$z = 1 \cdot x_1 + 1 \cdot x_2 - 1.5$

The step function turns the score into class 0 or 1.

Geometrically:

$x_1 + x_2 = 1.5$ is one straight line.

That line separates (1,1) from the other three points.

Why does OR succeed with one perceptron?

OR is also linearly separable.

Only one point should be class 0:

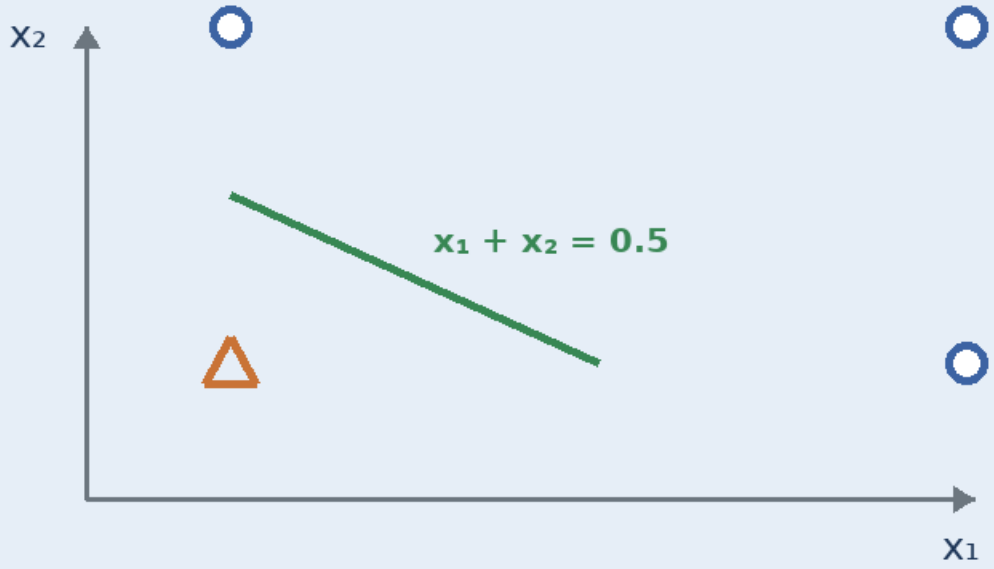
(0,0) → 0

All other points → 1

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

One straight line can isolate (0,0).

Again, one boundary is enough



Perceptron: $y = \text{step}(x_1 + x_2 - 0.5)$

Why AND and OR work, but XOR does not

AND

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

✓ **One line separates the classes**

Formula: step($x_1+x_2-1.5$)

OR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

✓ **One line separates the classes**

Formula: step($x_1+x_2-0.5$)

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

✗ **No single line can separate**

Needs hidden neurons + nonlinearity

Rule to remember: **A single perceptron works only when one straight line can separate the classes.**

The perceptron in three equations

1 SCORE $$s = \mathbf{w}^T \mathbf{x} + b$$

2 DECIDE $$\hat{y} = 1 \text{ if } s > 0; \quad \hat{y} = 0 \text{ otherwise}$$

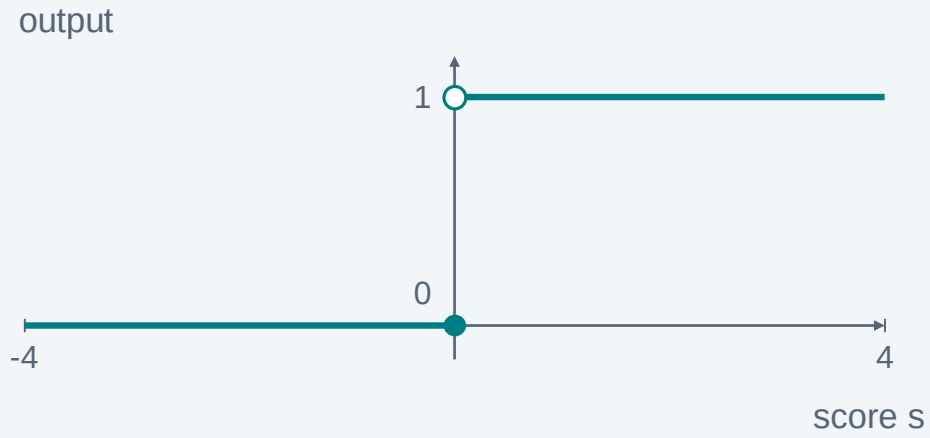
3 LEARN $$e = y - \hat{y}, \quad \mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}, \quad b \leftarrow b + \eta e$$

One perceptron learns one linear boundary in its supplied feature space.

Appendix A3 · From a hard decision to a smooth output

Start with the same score $s = w^T x + b$. The activation determines what comes out.

Step (hard threshold)

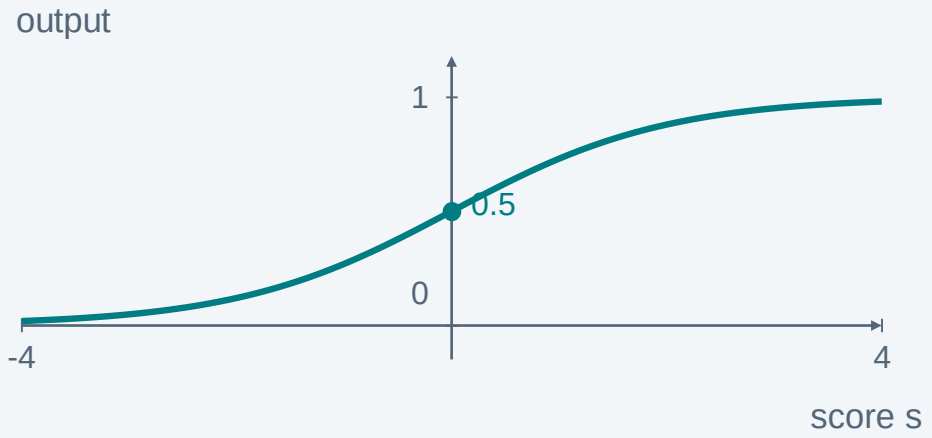


OUTPUT

Exactly 0 or 1: a class decision.

Choose for the classic perceptron.

Sigmoid (logistic)



OUTPUT

A value between 0 and 1.

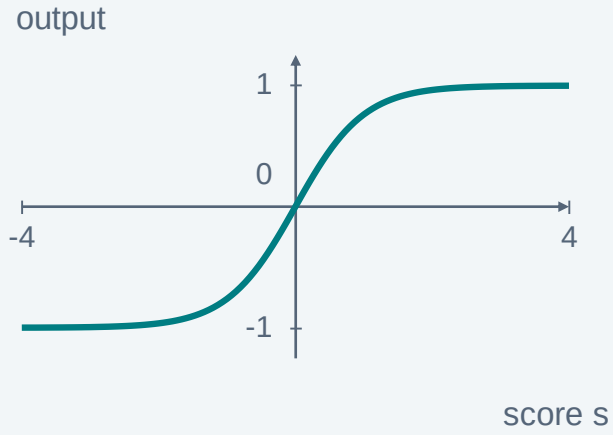
Choose for a binary-class probability output.

The classic perceptron uses the step function. The other activations preview later models.

Appendix A3 · Activation functions inside the network

A hidden layer is an intermediate processing stage, before the final prediction.

Tanh

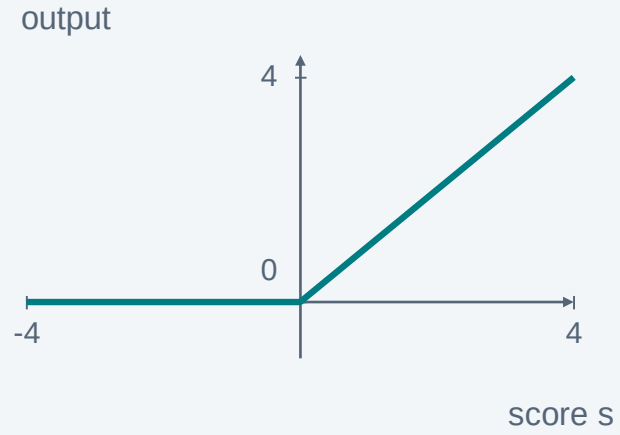


OUTPUT

Between −1 and +1.
A bounded, signed signal.

Use for a bounded signal centered at zero.

ReLU

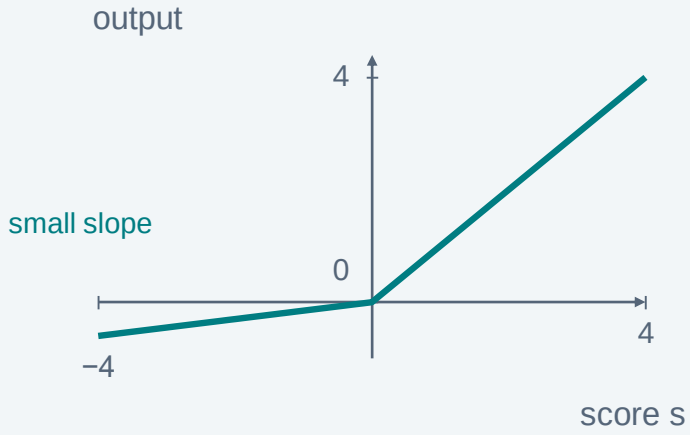


OUTPUT

Zero or positive.
Keeps positive scores.

A common choice for hidden layers.

Leaky ReLU



OUTPUT

Any real value.
Negatives are scaled down.

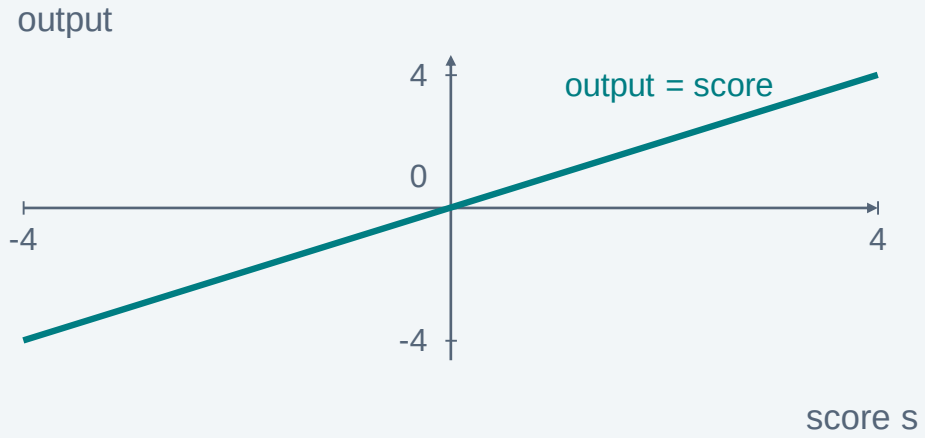
Keep learning on the negative side.

Hidden activations transform signals; they are not automatically class probabilities.

Appendix A3 · Choose the final output for the task

The output activation should match the kind of answer you need.

Linear (identity)

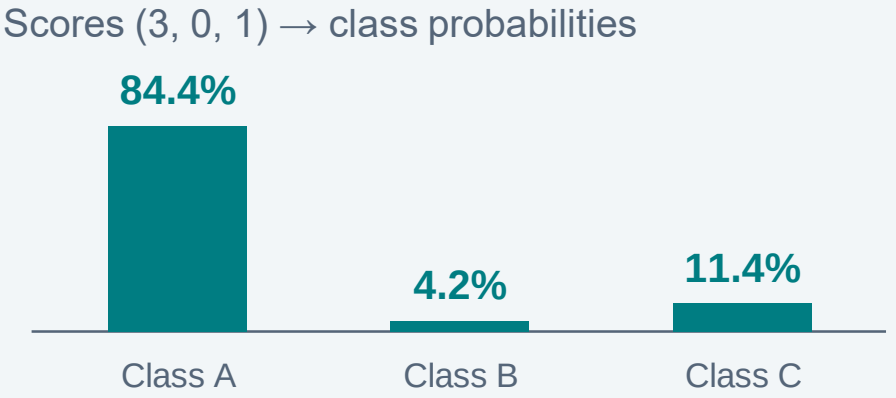


OUTPUT

Any real value; no squashing.

Choose for an unrestricted regression output.

Softmax



OUTPUT

One probability per class; total = 1.

Choose when only one class is correct.

Softmax acts on a group of scores not on one score in isolation.

Appendix A5 · Momentum changes one part of the update

Here w is a weight, and t is the update number. The bias can be updated in the same way.

WITHOUT MOMENTUM

The new weight change depends only on the current correction:

$$w_{t+1} = w_t + \Delta w_t$$

$$\Delta w_t = \eta e x_n$$

WITH MOMENTUM

We also keep part of the previous weight change

$$w_{t+1} = w_t + \Delta w_t$$

$$\Delta w_t = \eta e x_n + \alpha \Delta w_{t-1}$$

α Momentum Coefficient: fraction of previous memory retained
usually Choose 0 ≤ β < 1.

Learning rate controls the size of the current step.
Momentum remembers part of the previous step.